

Indian Institute of Technology, Jodhpur

Department Of CSE

PCS II

Lab: File System Exploration in xv6

Course Instructor: Dr Sidharth Sharma

Pre requisite-

- Make sure you have already downloaded and installed the **xv6 operating system source code** provided for the lab.

The operating system runs on an x86 emulator called **QEMU**.

In the xv6 directory run the following commands:

```
make
make qemu
```

These commands compile the xv6 source code and start the operating system inside the emulator.

- In this project you will explore the **file system implementation in xv6**.

Before starting the project you should study the following files:

```
fs.c
fs.h
file.c
file.h
sysfile.c
bio.c
log.c
```

-**fs.c** contains the implementation of the xv6 file system.

-**fs.h** defines file system structures such as **inodes and directory entries**.

-**file.c** implements file descriptor related operations.

-**sysfile.c** contains system calls related to file operations.

-**bio.c** implements the buffer cache used for disk access.

Understanding these files will help you understand how xv6 manages **files, directories, and disk blocks**.

- Learn how file related commands work in xv6.

Try running the following commands inside xv6:

```
ls
mkdir
touch
rm
cat
```

Observe how the file system behaves when files and directories are created or deleted.

Part A: Understanding the xv6 File System

In this part you will explore the structure of the xv6 file system.

Tasks:

1. Locate the definition of **struct inode** in the source code.
2. Identify the information stored in an inode.
3. Find how xv6 keeps track of **file size and data blocks**.
4. Study how directories store file names and corresponding inode numbers.
5. Trace the execution of a file related system call such as **open()** or **create()**.

Write a small program that creates a few files and directories and observe the changes using the `ls` command.

Part B: Implementing a File System Information System Call

In this part you will extend xv6 by implementing a system call that provides information about the file system.

The system call should return basic statistics about the current file system.

Examples of information that may be returned include:

- total number of files
- number of directories
- number of allocated inodes
- number of free disk blocks

Tasks:

1. Define a new system call in xv6.
2. Implement the system call inside the kernel.
3. Collect the required information from the file system structures.
4. Return the information to a user program.

Make sure the system call is properly registered in the syscall table.

Part C: Writing a User Program

Write a user program that calls the new system call and prints file system information.

The program should display information in a readable format.

Example output:

```
File System Information
-----
Total files : <value>
Total directories : <value>
Allocated inodes : <value>
Free disk blocks : <value>
```

Add the program to the **Makefile** so that it is compiled along with the other user programs.

Run the program inside xv6 and observe the output.

Hints

- Study how file related system calls such as **open()**, **read()**, and **write()** are implemented.
- The file system metadata is stored using **inodes and block bitmaps**.
- Directory entries map **file names to inode numbers**.
- Use **cprintf()** to print debugging messages if necessary.
- Look at the existing code that allocates and frees disk blocks.

Testing

After implementing the system call, test it using different scenarios.

You may try the following experiments:

1. Run the program when the system starts and observe the file system statistics.
2. Create multiple files and directories and run the program again.
3. Delete some files and check whether the statistics change correctly.

Verify that the information returned by the system call reflects the actual state of the xv6 file system.